

Welcome Test driven development TDD

discussion 

What is a "Unit"?

Unit

Smallest testable part of the system

Unit test

Smallest testable part tested in isolation (individually and independently)

Keeping tests clean

"Test code is just as important as production code. It is not a second-class citizen. It requires thought, design, and care. It must be kept as clean as production code."

—Robert C. Martin

Why Unit test?

The point of writing automated unit tests is not so much to verify that the code works **now**.

The point is to verify on an ongoing basis that the code **continues to work in the future**.

What do we test?

Behaviour

A test is not a unit test when:

- It talks to the database
- It communicates across the network
- It touches the file system or reads config info
- It uses `DateTime.now()` or `Random`
- It can't run at the same time as any of your other unit tests
- You have to do special things to your environment (such as editing config files) to run it.

Single assert per test

- Strive towards only having single assert
- Each test should verify only one thing.
- Don't try to test several things in the same test.
- It helps you have tests that fail for a specific reason, and drives you to focus on a specific behavior at a time.
- It's not the rule what really matters, is the intention behind the rule

Assertion

- `assertTrue(booleanCondition)`

Asserts that the condition is true

- `assertFalse(booleanCondition)`

Asserts that the condition is false

- `assertEquals(expected, actual)`

Assert that expect and actual are equal

- `assertArrayEquals(expectedArray, actualArray)`

Verifies each item in the array are equal in the right position

Asserting Exception

Two ways to assert on Exception

- `@Test(expected = RuntimeException.class)`

Verifies and expect an exception to be thrown anywhere in the annotated test

- `@Rule`

```
public ExpectedException expectedException = ExpectedException.none(); //class level
```

```
expectedException.expect(RuntimeException.class);  
expectedException.expectMessage("expected message");
```



inside the test, just
before the act statement

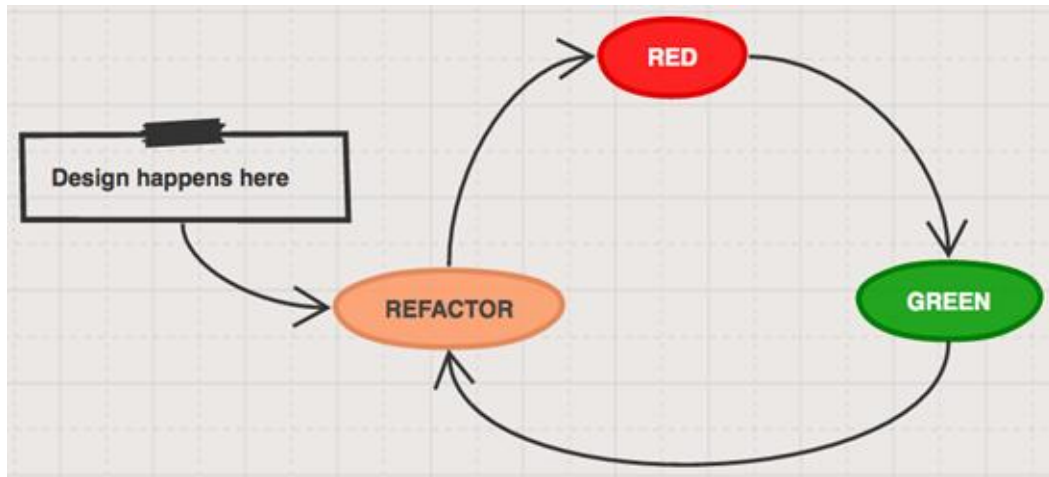
Test Driven Development

[Classical Style]

Classicist TDD

State based test, write a test, get a state and assert on state

Good for exploration



Examples

```
@Test
public void test() {
    assertEquals("cba", Util.reverse(input));
}
```

```
@Test
public void test() {
    Component component = new Component();
    component.setData("some data");
    assertEquals("some data", component.getData());
}
```

Discipline

- You cannot write any production code until you have a failing test
[Not compiling is failing]
- You are not allowed to write more in a unit test than is sufficient to fail, as soon as the unit test fails or fails to compile, you must stop writing test and start writing production code
- You are not allowed to write more production code than is sufficient to pass currently failing test

Discipline

- Many developers feel the urge to predict where the changes to the code should go - resist this urge. The beauty of TDD is that as you go along you will start to introduce the necessary features for the desired solution, but you will be guided by the tests, and the tests will confirm those changes once they are in place.
- **Be Patient.** To an experienced developer, these steps seem trivial and unnecessary. And for developers who are very experienced in TDD, they sometimes skip these steps. But any discipline is about learning to do it properly, and practising in this way until it becomes internalised and second-nature. When you're learning to play the piano, you need to practice your scales and chopsticks. It's the same with TDD.

TDD Steps

Simple 4 step process

- Test
 - write a test
- Fail
 - make it fail
- Run
 - make it run
- Refactor
 - remove duplications

Write a test

The first thing we do is: we write a test!

- Write unit test you want to read
- Isolate the first problem
- Make it concrete
- Express it in code (unit test behaviour not methods)
 - focus on the interface you design
 - focus on the behaviour you expect

Fail

Next: Run it and watch it fail

- We do not write any code yet
- we expect missing definitions
- We expect missing behaviour
- It must only be a few failures

A RED BAR IS PROGRESS !!!

Run

Next: We make it pass !

- Step by step we
 - Add code
 - Run the test and see less (or different) failures
- Simplicity
 - Just make it pass - keep it very simple
 - Cheating (e.g. returning constants) is allowed

Hurray! - green bar! - have a party!

Green bar patterns

Fake it ('til You Make It)

```
@Test
public void test() {
    assertEquals(10, sum(4, 6));
}

private int sum (int firstNumber, int secondNumber) {
    return 10;
}
```

Cheating!! (test pass)



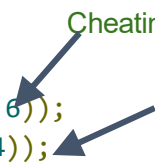
Triangulation

```
@Test
public void test() {
    assertEquals(10, sum(4, 6));
    assertEquals(8, sum(4, 4));
}

private int sum (int firstNumber, int secondNumber) {
    return 10;
}
```

Cheating!! (test pass)

failing test

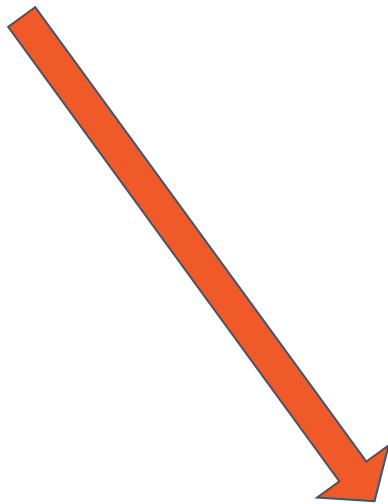


Refactor

Finally: We remove duplications!

- With the green bar we can
 - Clean up code
 - Remove duplications
- All tests should pass

/labs/day-8



We will drive the implementation with our first test. Calculate sum of two digits